

When Imbalance Correction Backfires

A Plain-Language Guide to the Project — What We Set Out to Answer, How We Built the Test, and What We Found

Companion explainer for the research study "When Imbalance Correction Backfires: Robustness of Tabular Imbalance Mitigation Under Minority-Label Contamination"

Written for understanding, not for citation — every number in this document traces back to the project's verified results files (`results/`, `tables/`) or a clearly marked illustrative example.

Contents

1. The One-Paragraph Version
2. The Problem: Why This Project Exists
3. Background You Need First
4. How We Built a Fair Test, Step by Step
5. The Correction Methods We Tested (with formulas and worked examples)
6. How We Measured "Did It Help or Hurt?" (with formulas and worked examples)
7. What We Actually Found
8. What This Project Is Trying to Tell You
9. Glossary of Terms

1. The One-Paragraph Version

When one class in your data is much rarer than the other (say, 5% fraud vs. 95% legitimate transactions), it's standard practice to apply an "imbalance correction" — a technique that nudges the model to pay more attention to the rare class. Everybody assumes these corrections are basically safe: they might not always help a lot, but they shouldn't hurt. This project tested that assumption under a condition nobody had carefully tested before: what happens when the labels you're correcting *for* are themselves partly wrong? We built a controlled experiment across 5 real datasets, 2 kinds of model (TabPFN, a newer AI model, and XGBoost, a classic one), 7 different correction techniques, and 2 distinct ways labels can be wrong, and measured — precisely and reproducibly — how often each correction technique actually made things worse instead of better. The answer, backed by statistics, is: often enough that "just apply a correction" is not a safe default. Some corrections fail more than 30% of the time under one kind of label error and less than 1% of the time under the other. Which correction you pick, and which direction your label errors run, both matter enormously.

2. The Problem: Why This Project Exists

Imagine you're building a model to detect fraud. In your training data, fraud is rare — maybe 500 fraud cases out of 10,000 transactions (5%). If you train a normal model on this as-is, it will often learn to just predict "not fraud" almost every time, because that's right 95% of the time and requires no real understanding of fraud. This is **class imbalance**, and it's one of the most common problems in applied machine learning.

PLAIN LANGUAGE

Think of it like studying for a test where 95% of the practice questions are about Chapter 1 and only 5% are about Chapter 2. You'll get really good at Chapter 1 and barely learn Chapter 2 at all — even if Chapter 2 questions are just as important on the real exam.

The standard fix is an **imbalance-correction technique**: something that makes the model pay more attention to the rare class. There are many such techniques — duplicating rare examples, generating synthetic rare examples, penalizing mistakes on the rare class more heavily, or (for a newer kind of model) mathematically adjusting the model's output probabilities after the fact. Every practitioner's textbook treats these as close to a free lunch: apply one, and your model gets better at the rare class, typically without giving up much on the common class.

But there's an assumption baked into every one of these techniques that almost nobody questions: **the labels are correct**. The correction techniques work by trusting the observed count of "rare class" examples — if the technique thinks there are 500 fraud cases, it calibrates its correction around that number. What if 100 of those "fraud" labels are actually mistakes — legitimate transactions someone mislabeled? Or what if 100 real fraud cases got mislabeled as legitimate, and are hiding in the "not fraud" pile? In the real world, labels are never perfect. Human annotators disagree, automated labeling pipelines have bugs, and rare-class labels in particular are disproportionately prone to error because there are so few of them to begin with.

PLAIN LANGUAGE

Going back to the studying analogy: what if some of the "Chapter 1" practice questions were secretly mislabeled Chapter 2 questions, or vice versa? A study plan built by trusting those labels could send you in exactly the wrong direction — and you'd have no way to know just from your practice-test score.

This project's central question is simple to state and, as far as we could establish, had not been carefully answered before: **when a controlled amount of label error is mixed in with class imbalance, do the standard corrections still help — or can they actively**

make the model worse than if you'd done nothing at all? And if they can backfire, under what conditions, and how badly?

3. Background You Need First

3.1 The two kinds of models we tested

XGBoost is a well-established, classic machine learning method. It builds hundreds of small decision trees one after another, where each new tree tries to fix the mistakes of the trees before it. It has to be "trained" — shown the training data and adjusted through many rounds of calculation — before it can make a single prediction. It's fast, reliable, and used constantly in industry.

TabPFN is a newer, different kind of model. Instead of being trained from scratch on your specific dataset, it was pretrained once (by its creators) on millions of synthetic example datasets, and it makes predictions by treating your entire training set as part of its "prompt" — similar in spirit to how a large language model reads context before answering a question. This is called **in-context learning**. It needs no training time on your dataset and often performs excellently on small-to-medium sized tables of data. Because it works so differently from XGBoost, the class-imbalance corrections designed for it are also different — some work by adjusting its predicted probabilities directly, after it has already made its prediction.

3.2 What "label noise" means

Label noise just means: some of the labels in your dataset are wrong. This project focuses on a specific, realistic kind of label noise: **directed** noise, where the mistakes systematically flow in one particular direction rather than being totally random. We test two distinct directions:

- **Direction A (majority → minority contamination):** some truly-majority-class rows get mislabeled as belonging to the minority class. Example: a normal transaction gets incorrectly flagged as fraud.
- **Direction B (minority → majority loss):** some truly-minority-class rows get mislabeled as belonging to the majority class — the rare cases get "lost" into the common pile. Example: a real fraud case slips past review and gets recorded as legitimate.

PLAIN LANGUAGE

Direction A is like a jumpy fraud filter that cries wolf on innocent transactions. Direction B is like a lazy reviewer who waves real fraud through. Both are "5% wrong labels," but they distort the data in opposite ways — and, as this project shows, they affect corrections very differently.

3.3 The five datasets

We used five real, publicly available datasets from the UCI Machine Learning Repository, chosen so results wouldn't be an artifact of just one dataset's quirks:

Dataset	Rows	Features	What "minority class" means here
Banknote Authentication	1,372	4	Forged/counterfeit banknote
Blood Transfusion	748	4	Donated blood in the target time period
South German Credit	1,000	20	Bad credit risk
Spambase	4,601	57	Spam email
Breast Cancer Wisconsin	569	30	Malignant diagnosis

4. How We Built a Fair Test, Step by Step

1 Split each dataset once, cleanly. Before doing anything else — no imbalance manipulation, no label errors — we split each dataset 80% training / 20% testing, keeping the same class ratio in both pieces (this is called a *stratified split*). The test set is set aside and never touched again for that run. This guarantees every method is graded on the exact same, clean, untouched exam.

2 Fix the total training-set size across every imbalance level — the "N_common" trick. This is one of the more subtle design choices, and worth explaining carefully, because it's what makes the whole comparison fair.

PLAIN LANGUAGE

If you compare a model trained on "1,000 rows, 20% minority" against a model trained on "300 rows, 5% minority," and the second model does worse, you can't tell if that's because of the lower minority *percentage*, or simply because it had less data overall. To isolate imbalance as the one thing being varied, every imbalance level tested needs to use the exact same *total* number of training rows.

We compute one shared number, called `N_common`, which is the largest training-set size that is achievable at *every* imbalance ratio we want to test, given how many minority and majority rows actually exist in the training split.

$$\text{feasible_n}(r) = \min \left(\frac{n_{\text{min}}}{r}, \frac{n_{\text{maj}}}{1 - r} \right)$$

$$N_{\text{common}} = \lfloor \min_{r \in \text{tested ratios}} \text{feasible_n}(r) \rfloor$$

`n_min` = minority rows available in this training split · `n_maj` = majority rows available

`r` = target minority ratio (e.g. 0.20) · `⌊ x ⌋` = round `x` down to a whole number

WORKED EXAMPLE — REAL NUMBERS FROM THIS STUDY

For the Banknote dataset, seed 13, the training split has **488 minority rows** and **609 majority rows**. We test four imbalance ratios: `r = 0.50, 0.20, 0.10, 0.05`.

r	Limited by minority (488 / r)	Limited by majority (609 / (1-r))	feasible_n(r)
0.50	976.0	1218.0	976.0
0.20	2440.0	761.2	761.2

0.10	4880.0	676.7	676.7
0.05	9760.0	641.1	641.1

The tightest constraint across all four ratios is 641.1 (from $r = 0.05$), so **$N_{\text{common}} = \text{floor}(641.1) = 641$** . Every one of the four imbalance conditions for Banknote, seed 13, uses exactly 641 training rows — only the minority/majority split within those 641 rows changes.

3 Sample the controlled training set for each imbalance ratio. For a given ratio r and N_{common} , we take `round( $r \times N_{\text{common}}$ )` minority rows and the rest majority rows, drawn randomly without replacement, then shuffle them together.

WORKED EXAMPLE (CONTINUED)

At $r = 0.20$ and $N_{\text{common}} = 641$: minority rows needed = `round( $0.20 \times 641$ )` = **128**, majority rows needed = $641 - 128 = \mathbf{513}$.

4 Inject the directed label error, at a controlled rate c . This is the heart of the experiment. We test five noise levels: $c = 0.00$ (clean, no corruption), 0.05, 0.10, 0.20, and 0.30, in each of the two directions described in Section 3.2.

Direction A — majority rows get mislabeled as minority

We want the *final observed* fraction of fake-minority labels among all "minority-looking" rows to equal c . If k is the number of majority rows we flip to look like minority rows, and n_{min} is the true minority count, then we need:

$$c = \frac{k}{n_{\text{min}} + k} \quad \text{solved for } k: \quad k = \frac{c}{1 - c} \times n_{\text{min}}$$

Direction B — minority rows get mislabeled as majority (lost)

Here it's simpler: c is directly the fraction of true minority rows we erase.

$$k = c \times n_{\text{min}}$$

WORKED EXAMPLE (CONTINUED) — $C = 0.20$

At $r = 0.20$, we have $n_{\text{min}} = 128$ minority rows.

Direction A: $k = \text{round}((0.20 / 0.80) \times 128) = \text{round}(32.0) = \mathbf{32}$ majority rows get relabeled as minority. Check: $32 / (128 + 32) = 32 / 160 = \mathbf{0.200}$ — exactly the target rate.

Direction B: $k = \text{round}(0.20 \times 128) = \text{round}(25.6) = \mathbf{26}$ minority rows get relabeled as

majority (lost). Check: $26 / 128 = \mathbf{0.203}$ — very close to the target (small rounding gap, an unavoidable side effect of needing a whole number of rows).

5 Preprocess safely, without leaking information. Any imputation (filling in missing values) or encoding is calculated *only* from the current corrupted training set, then applied to both that training set and the untouched test set. The test set is never used to decide how to fill in a missing value — that would let information "leak" backwards from the exam into the study material.

6 Run every correction method on this exact same corrupted training set, and score every one against the same clean test set. This is repeated for 3 different random seeds (13, 37, 73), so results aren't a fluke of one particular random split or one particular set of flipped rows.

PUTTING IT ALL TOGETHER — FROM 540 STRESS CELLS TO 6,417 RESULT ROWS

A **stress cell** is one fully-specified test condition: one dataset, one seed, one imbalance ratio, one noise condition. Every method then gets run on that exact same cell, so results are directly comparable.

Factor	Count	What it is
Datasets	5	Banknote, Blood Transfusion, South German Credit, Spambase, Breast Cancer
Seeds	3	13, 37, 73 — repeats every condition with a different random split/sample
Imbalance ratios (r)	4	0.50, 0.20, 0.10, 0.05
Noise conditions	9	1 clean (c=0.00) + 4 noise levels × 2 directions

Why 9, not 10? You might expect $4 \text{ noise levels} \times 2 \text{ directions} = 8$, plus the clean condition = 9 — but not $1 \times 2 + 4 \times 2 = 10$. At $c = 0.00$ there is zero contamination, and "which direction did the contamination go" is a meaningless question when nothing was corrupted. So the clean condition counts once, not twice.

$5 \text{ datasets} \times 3 \text{ seeds} \times 4 \text{ ratios} \times 9 \text{ conditions} = \mathbf{540 \text{ stress cells}}$

From cells to rows: each stress cell doesn't produce one row — it produces one row *per method* run on it.

Family	Methods run per cell	Rows
XGBoost	xgb_none, xgb_weighted, xgb_ros, xgb_smote	4
TabPFN	tabpfn_none (1); tabpfn_threshold, tabpfn_distpfn, tabpfn_distpfn_t — each scored twice, once with the realistic <i>observed</i> prior and once with an	8

	unrealistic <i>oracle</i> prior that cheats by knowing the true pre-corruption rate ($2 \times 3 = 6$); <code>tabpfn_downsample (1)</code>	
--	---	--

540 cells $\times$ 12 rows/cell = **6,480 possible rows**

Why 6,417, not 6,480? The gap is exactly $6,480 - 6,417 = 63$ — the number of documented `tabpfn_downsample` failures (the $r=0.50$ rounding edge case: after corruption, the "minority" count can end up numerically larger than the "majority" count, leaving nothing left to downsample). Those 63 cells still succeed for every *other* method — only that one method's row is missing, and it's recorded in `failures.csv` rather than silently dropped.

6,480 possible – 63 known, explained failures = **6,417 actual result rows**

5. The Correction Methods We Tested

We split correction methods into two families, one per model type, so every method is compared fairly against the plain, uncorrected version of *the same model*.

5.1 XGBoost family

xgb_none (the baseline — no correction at all)

Just train XGBoost normally on the corrupted, imbalanced data — no imbalance handling of any kind. This matters as more than "the boring option": it's the reference point every other method has to beat. Every result in this study is stated as *Correction Gain* — the difference between a corrected method's score and `xgb_none`'s score at the exact same stress condition. Without it, there would be nothing to measure "did the correction actually help" against.

Left alone, XGBoost trains by minimizing average loss across all training rows. With, say, 513 majority rows and 160 minority-looking rows, the model can get 76% of its loss "right" just by leaning toward predicting majority almost everywhere — it never gets forced to pay real attention to the rare class.

xgb_weighted (cost-sensitive reweighting)

Instead of changing the data, we tell the model "a mistake on a minority row costs more than a mistake on a majority row," in direct proportion to how imbalanced the observed data is.

$$\text{weight} = \frac{\text{observed majority count}}{\text{observed minority count}}$$

What this actually does inside training: XGBoost's `scale_pos_weight` parameter doesn't touch the data at all — it changes how mistakes are *scored*. At every boosting round, XGBoost computes a gradient/loss contribution for every row; `scale_pos_weight` multiplies the contribution of every minority-labeled row by that weight. A weight of 3.21 means a wrong prediction on a minority row pushes the model to correct itself roughly 3.21× as hard as a wrong prediction on a majority row.

WORKED EXAMPLE

513 majority rows, $128 + 32 = 160$ "minority-looking" rows (after Direction A corruption at $c=0.20$): $\text{weight} = 513 / 160 \approx \mathbf{3.21}$. The model is told each mistake on a minority-looking row counts as if it were 3.21 mistakes.

WHERE THE BACKFIRE RISK COMES FROM

The weight is computed from the *observed*, possibly-corrupted count (160), not the true count (128). Under Direction A, 32 of those 160 rows are actually mislabeled majority rows — the formula has no way to know that; it just sees "160 things that look minority." The resulting weight (3.21) ends up *lower* than what the true count alone would give ($513/128 \approx 4.01$), because the fake rows inflate the denominator. The correction under-weights the real minority signal relative to what it "should" under clean labels — a miscalibration baked directly into the formula itself, not a separate bug.

xgb_ros (Random Over-Sampling)

Before training, `RandomOverSampler` looks at the imbalanced training set and randomly duplicates minority-looking rows — picking existing rows *with replacement* — until both classes have the same count. With 160 minority-looking rows and 513 majority rows, it needs 353 more copies: it might draw row 4 twice more, row 91 once more, row 12 five times more, and so on, purely at random, until the minority-looking pile also reaches 513. XGBoost then trains normally on the now-balanced 1,026-row set.

WHERE THE BACKFIRE RISK COMES FROM

ROS treats every minority-looking row as equally trustworthy when deciding what to duplicate — it has no way to tell the 32 fake rows apart from the 128 real ones, so it draws duplicates from the whole pool uniformly at random. Statistically, roughly 20% of every duplicate it creates ($32/160$) is a copy of a *fake* row. The correction doesn't just fail to filter out the contamination — it actively **multiplies** it, planting several exact copies of wrong information throughout the training set.

xgb_smote (Synthetic Minority Over-sampling)

Rather than duplicating existing rows exactly, SMOTE invents brand-new ones. For each synthetic row it wants to create:

1. Pick a real minority-looking row, call it A.
2. Find its k nearest minority-looking neighbors in feature space (this study uses $k = \min(5, n_{\text{min}} - 1)$, capped so it never asks for more neighbors than exist).
3. Randomly pick one neighbor, B, from those k .
4. Pick a random number t between 0 and 1.
5. Create a new row at $A + t \times (B - A)$ — a point on the line between A and B.

WORKED EXAMPLE (2 FEATURES, FOR SIMPLICITY)

Say $A = (2.0, 5.0)$ and its neighbor $B = (4.0, 7.0)$, and $t = 0.3$ is drawn at random.

$$\begin{aligned}\text{new row} &= (2.0, 5.0) + 0.3 \times [(4.0, 7.0) - (2.0, 5.0)] \\ &= (2.0, 5.0) + 0.3 \times (2.0, 2.0)\end{aligned}$$

$$\begin{aligned} &= (2.0, 5.0) + (0.6, 0.6) \\ &= \mathbf{(2.6, 5.6)} \end{aligned}$$

That point — 30% of the way from A toward B — becomes a new "minority" training row, with a feature vector that never existed in the original data.

WHERE THE BACKFIRE RISK COMES FROM — AND WHY IT'S WORSE THAN ROS IN ONE SPECIFIC WAY

SMOTE has the same underlying vulnerability as ROS: a fake row can absolutely get picked as A or B, just as easily as a real one. But it adds a second failure mode on top: even when A is a real minority row, if its nearest neighbor B happens to be one of the fake rows sitting nearby in feature space, SMOTE generates a synthetic point *between* a real example and a fake one — manufacturing an entirely new row that doesn't cleanly belong to either. It isn't just duplicating contamination like ROS; it's actively **blending** contamination into brand-new synthetic points, potentially spreading the fake signal into regions of feature space no original fake row ever occupied. This is a real part of why Section 7's Finding 2 shows `xgb_smote`'s CHR (25.0% under Direction A) running noticeably higher than `xgb_weighted`'s (15.0%) — weighting only miscalibrates a single number, while ROS and SMOTE physically inject corrupted or corruption-derived rows into the training set itself, a structurally larger vulnerability under exactly the kind of label contamination this study tests.

5.2 TabPFN family

How TabPFN actually works

XGBoost and TabPFN are fundamentally different kinds of models, and the difference matters for understanding every correction method below. With XGBoost, calling `.fit(X_train, y_train)` builds and adjusts a bespoke model — hundreds of decision trees — specifically for your dataset, through many rounds of calculation. A model trained on Spambase knows nothing that transfers to Banknote.

TabPFN works completely differently, in two separate phases.

Phase 1 — pretraining (done once, years ago, by TabPFN's creators, before you ever touch it): they generated millions of synthetic, fake tabular datasets — small random classification problems with made-up features and made-up relationships between those features and a label, drawn from a "prior" that encodes general assumptions about what real-world tabular data tends to look like. Because they generated each synthetic dataset themselves, they know the mathematically correct answer for it. They trained one large transformer neural network to take "(a batch of training rows and labels, plus one query row)" as input and output the correct predicted probability, repeated across millions of these fake datasets. This teaches the network a general *skill* — how to look at a set of labeled examples and infer the right answer for a new one — not facts about any particular dataset, since it never saw yours.

Phase 2 — inference (what happens when you call TabPFN on your actual data):

`.fit(X_train, y_train)` runs no training, gradient descent, or weight updates at all — the network's weights are already frozen from Phase 1; this call just stores your rows.

`.predict_proba(X_test)` is where the real work happens: your entire training set and the test row you want predicted are fed into the transformer *together*, in one single forward pass. The transformer's attention mechanism lets the test row "look at" every training row at once, weigh how relevant each one is, and output a predicted probability.

PLAIN LANGUAGE

This is called **in-context learning** — the same trick large language models use when you give them a few examples in the prompt before asking a question. TabPFN treats your training set as its "prompt" and your test row as the "query." It never permanently learns anything from your data; it re-reads your entire training set fresh, every single time, for every prediction.

Why this matters for every method below: since TabPFN never actually trains on your specific dataset, everything it "knows" about class balance comes from two places — whatever general expectations about class distributions got baked in during the millions of synthetic pretraining tasks, and whatever it can infer, in a single forward pass, from the training rows you hand it right now. When your training rows are imbalanced or corrupted, that mismatch between the pretraining-derived expectation and your actual data shows up directly in the raw output probabilities — which is exactly the gap `tabpfn_threshold`, `tabpfn_distpfn`, and `tabpfn_distpfn_t` are each trying to correct, in different ways.

`tabpfn_none` (the baseline)

Just ask TabPFN for its raw predicted probability that each test row belongs to the minority class, and predict "minority" whenever that probability exceeds 0.5. Same role as `xgb_none` : the reference every other TabPFN method has to beat.

`tabpfn_threshold`

This one doesn't touch the probability at all — it's a purely *decision-rule* change, made after the forward pass, with zero extra computation. Instead of using the standard 0.5 cutoff, shift the decision boundary down to match the observed minority rate. If minority rows are rare, the model doesn't need to be "more than half sure" to guess minority — it just needs to be more confident than the base rate would suggest by chance.

$$\text{predict minority if } P(\text{minority}) > \tau$$

where τ is the observed minority fraction in the training data (e.g. 0.20 for a 20%-minority training set).

WHERE THE BACKFIRE RISK COMES FROM

τ is computed from the *observed*, possibly-corrupted training prior, not the true one. Under Direction A (majority rows faked into minority), the observed minority count is inflated, so τ ends up *higher* than the honest rate would justify. Under Direction B (minority rows lost into majority), the observed count shrinks, so τ ends up *lower* than it should. Either way, the threshold is calibrated against a number that's already wrong by exactly the amount of contamination present — a real correction, built on a foundation it has no way to verify.

tabpfn_distpfn and tabpfn_distpfn_t

These two are more involved, and they're specific to TabPFN's kind of model. They come from a 2026 paper (Lee, ICML 2026) that we validated directly against the authors' own published code before using it — and in doing so, we actually found and had to correct a mistake in an earlier draft of our own implementation (see the note at the end of this section).

The idea: TabPFN's raw predicted probabilities are shaped by whatever class balance it implicitly "expects" from its pretraining. DistPFN rescales those probabilities to correct for the mismatch between that expectation and the training data's observed class balance.

$$\begin{aligned} p &= [P(\text{majority}), P(\text{minority})] \quad (\text{TabPFN's raw prediction for one test row}) \\ \bar{p} &= \text{average of } p \text{ across every row in the test batch} \quad (\text{one shared vector for the whole batch}) \\ \text{prior} &= [1 - \pi, \pi] \quad (\pi = \text{observed minority fraction}) \\ \text{adjusted} &= \text{Normalize}(p \times \bar{p} / \text{prior}) \end{aligned}$$

WORKED EXAMPLE (ILLUSTRATIVE NUMBERS, TO SHOW THE MECHANICS)

Suppose we have just 2 test rows, and TabPFN's raw prediction for "minority" is 0.30 for row 1 and 0.70 for row 2. As 2-class vectors (majority first, minority second): row 1 = [0.70, 0.30], row 2 = [0.30, 0.70]. The batch average is $\bar{p} = [0.50, 0.50]$. Suppose the observed minority prior is $\pi = 0.20$, so prior = [0.80, 0.20].

Row 1 — each entry is divided by its *own* position's prior, not the same prior for both: adjusted (before normalizing) = [0.70×0.50 / 0.80, 0.30×0.50 / 0.20] = [0.4375, 0.75]. Sum = 1.1875. Normalized = [0.368, **0.632**].

Row 1's minority probability moved from a raw **0.30** to a corrected **0.632** — more than double — because the minority prior (0.20) is much smaller than the majority prior (0.80), so dividing by it inflates any row that already leans toward minority.

Row 2 (same steps): adjusted = [0.1875, 1.75], sum = 1.9375, normalized = [0.097, **0.903**] —

row 2's minority probability moves from a raw **0.70** up to **0.903**, an even larger boost, for the same reason. Both rows get pushed further toward minority once the correction is applied — this is DistPFN actively compensating for a prior it treats as under-representing the minority class; whether that push helps or hurts depends on whether the prior it's compensating against is trustworthy, which is exactly what breaks down once label noise is present (Section 7).

DistPFN-T adds one more refinement: instead of using the raw batch average $\bar{p}$ directly, it first "sharpens" or "softens" it with a single shared temperature value τ , based on how different the batch average already is from the prior — the goal being to avoid over-correcting when the two already roughly agree.

$$\begin{aligned}\tau &= -\sum_i \bar{p}_i \cdot \log(\text{prior}_i) \quad (\text{cross-entropy between } \bar{p} \text{ and prior} - \text{one shared number}) \\ \bar{p}_T &= \text{softmax}(\bar{p} / \tau) \quad (\text{temperature-scaled version of } \bar{p}, \text{ still a shared vector}) \\ \text{adjusted} &= \text{Normalize}(\bar{p} \times \bar{p}_T / \text{prior}) \quad (\text{same last step as plain DistPFN, using } \bar{p}_T \text{ instead of } \bar{p})\end{aligned}$$

WORKED EXAMPLE — DISTPFN VS. DISTPFN-T ON THE SAME ROW

Suppose the batch average is $\bar{p} = [0.65, 0.35]$ this time (not perfectly balanced), with the same prior $= [0.80, 0.20]$, and we adjust the same row 1 prediction $p = [0.70, 0.30]$ as before.

Plain DistPFN uses $\bar{p}$ directly: adjusted minority probability = **0.480**.

DistPFN-T first computes $\tau = -(0.65 \times \log 0.80 + 0.35 \times \log 0.20) = \mathbf{0.708}$, then $\bar{p}_T = \text{softmax}([0.65, 0.35] / 0.708) = [\mathbf{0.604}, \mathbf{0.396}]$ — pulled slightly toward 50/50 compared to the raw $[0.65, 0.35]$, since dividing by $\tau < 1$ sharpens small gaps rather than shrinking them here. Using $\bar{p}_T$ in the same last step gives an adjusted minority probability of **0.529** — noticeably different from plain DistPFN's 0.480 for the exact same raw prediction and prior, purely because of the temperature step in between.

A FINDING WORTH KNOWING ABOUT, IN ITS OWN RIGHT

While preparing this study, we checked our DistPFN/DistPFN-T formulas directly against the authors' own published code on GitHub, rather than trusting our own reading of the paper's math. We found our first implementation had mistranscribed the formula — and, separately, that the authors' own published reference code for DistPFN-T contains a bug that makes it silently fail every time it's run for a two-class problem. We fixed our implementation to match the algorithm's clear intent rather than reproduce either mistake, documented exactly what we found and why, and only then generated the results in this study. This is exactly the kind of

error label-error-under-imbalance research needs to guard against — the same discipline we're applying to models, we also applied to our own code.

tabpfn_downsample

This one is structurally different from the other three — it's the only TabPFN correction that changes the *content* of the prompt itself, before TabPFN ever sees it, rather than adjusting the output afterward. It randomly drops majority rows until the majority count matches the minority count exactly, then TabPFN does its normal in-context forward pass on this now-balanced, smaller set.

WHERE THE BACKFIRE RISK COMES FROM — A DIFFERENT MECHANISM FROM ROS/SMOTE'S

Downsampling only removes *majority* rows; it never inspects or removes anything from the minority-looking pile, so every fake minority row (from Direction A contamination) survives completely intact. But here's the subtler part: because the total prompt shrinks down to $2 \times (n_{\min} + k)$ rows while the number of fake rows k stays fixed, **the fake rows become a larger proportion of the whole prompt after downsampling than they were before** — shrinking the majority pile doesn't dilute the contamination, it concentrates it. TabPFN ends up reading a smaller, but proportionally *more* contaminated, training set than it started with.

A related, separate limitation: because downsampling forces majority count to exactly equal minority count, it can occasionally have no valid majority pool left to draw from — this is the $r=0.50$ rounding edge case from Section 4, where contamination or rounding can push the observed "minority" count above the observed "majority" count, leaving nothing to downsample down to. That's the source of the 63 documented failures counted in Section 4.

6. How We Measured "Did It Help or Hurt?"

We recorded 10 different performance metrics for every single run (things like accuracy, precision, recall, and how well-calibrated the model's confidence was) — but the real question this project asks isn't "how good is the model," it's "did the *correction* help, compared to doing nothing, at this exact same stress level?" That required inventing four purpose-built comparison metrics.

Correction Gain (CG)

At a specific stress condition (same dataset, ratio, noise level, direction), how much better did the corrected method do than the uncorrected baseline, on the same model family?

$$\text{CG} = \text{score}(\text{corrected method}) - \text{score}(\text{uncorrected baseline})$$

Positive CG means the correction helped. Negative CG means it hurt.

Correction Harm Rate (CHR)

Across all the stress conditions we tested for one method, what *fraction* of the time was CG negative — i.e., the correction actually made things worse?

$$\text{CHR} = \frac{\text{number of stress cells where CG} < 0}{\text{total number of stress cells tested}}$$

A REAL RESULT FROM THIS STUDY

`tabpfn_downsample`'s CHR is **31.7%** under Direction A (majority→minority contamination) but only **7.5%** under Direction B (minority→majority loss). Same method, same overall noise levels tested — but it backfires more than 4× as often depending purely on which direction the label errors run.

Break-Even Point (c^*)

For a given method and dataset, as we crank up the noise level c from 0.00 upward, at what point does the correction stop helping and start hurting? This is the noise level c^* where CG first crosses from positive to negative.

A REAL RESULT FROM THIS STUDY

For `tabpfn_distpfn` on the Banknote dataset under Direction A, the break-even point is $c^* = 0.05$ — the very first noise level we tested. That means this correction is *already* net-harmful with just 5% label contamination on this dataset. On Breast Cancer Wisconsin under the same direction, the break-even point is > 0.30 — it never turned harmful across our whole tested range. Same method, wildly different robustness depending on the dataset.

Prior Sensitivity Gap (PSG)

Some corrections (threshold, DistPFN, DistPFN-T) need to know the minority class's "true" rate to calibrate themselves. In real deployment you only ever have the *observed* (possibly corrupted) rate — you can't know the true, pre-corruption rate. PSG measures how much better the correction would have done if it had magically been given the true rate instead of the observed one.

$$\text{PSG} = \text{score}(\text{using the true, oracle prior}) - \text{score}(\text{using the observed prior})$$

A small PSG is reassuring: it means not knowing the true rate isn't costing you much. This study found PSG values were small and sometimes even slightly negative — meaning the "cheat" of knowing the true rate barely helped, or occasionally hurt slightly, which is itself an interesting, non-obvious result.

7. What We Actually Found

FINDING 1 — THE DIFFERENCES ARE REAL, NOT NOISE

We ran a statistical test (the Friedman test) comparing every method within each model family and direction. Every single comparison came back extremely statistically significant ($p < 0.0000000001$ in every case) — the differences between methods are not random chance, they are real and repeatable.

FINDING 2 — CORRECTION HARM RATE VARIES ENORMOUSLY BY METHOD AND DIRECTION

Method	CHR, Direction A	CHR, Direction B
tabpfn_threshold	11.9%	0.6%
tabpfn_distpfn	23.1%	28.8%
tabpfn_distpfn_t	22.5%	25.0%
tabpfn_downsample	31.7%	7.5%
xgb_weighted	15.0%	10.0%
xgb_ros	18.8%	12.5%
xgb_smote	25.0%	10.0%

No method is safe across the board. Some methods (like `tabpfn_threshold`) are fairly robust to one kind of label error but not the other. Others (`tabpfn_distpfn`) are middling-to-risky in both directions. `tabpfn_downsample` is the most direction-sensitive of all: it backfires nearly a third of the time under one kind of corruption and less than a tenth of the time under the other.

FINDING 3 — HOW MUCH PERFORMANCE DROPS AS NOISE INCREASES

Balanced accuracy for the plain, uncorrected XGBoost baseline drops from **0.779** (clean data) to **0.749** under Direction A and **0.695** under Direction B, at the highest noise level we tested ($c = 0.30$), averaged across all 5 datasets. Direction B — where real minority-class examples get lost into the majority pile — is consistently the more damaging direction across nearly every method we tested.

FINDING 4 — ROBUSTNESS DEPENDS ON THE DATASET, NOT JUST THE METHOD

Break-even points for the same method swing from as low as $c^* = 0.05$ on one dataset to beyond $c^* = 0.30$ (never breaking even within our tested range) on another. A correction that's essentially bulletproof on one kind of data can be fragile on another — which means "is this

correction safe?" doesn't have one universal answer; it depends on the specific dataset's structure too.

8. What This Project Is Trying to Tell You

The title of this study is deliberately provocative: "*When Imbalance Correction Backfires.*" The point isn't that imbalance corrections are bad — they clearly help in the clean, no-label-error case, which is the only case most benchmarks ever test. The point is narrower and, we think, more useful in practice:

1. **"Apply a correction" is not automatically safe.** Once any realistic amount of label error is mixed into the picture — and real-world labels are essentially never perfectly clean — some widely-used corrections can make a model measurably worse than doing nothing at all, and this happens often enough (up to roughly a third of tested conditions for the worst offenders) that it isn't a rare edge case.
2. **The direction of the label error matters as much as the amount.** A model being defensive (Direction A: too many false alarms) and a model being complacent (Direction B: missing real cases) are not mirror images of each other from a correction's point of view — several corrections handle one direction far better than the other.
3. **There is no single "safest" correction across the board.** The method that's most robust depends on which kind of label error you're worried about, and even on which dataset you're working with.
4. **Practical takeaway for anyone applying these techniques:** before trusting an imbalance correction in a system where label quality isn't guaranteed, it's worth testing how that correction behaves under a *range* of plausible label-error rates and directions — not just checking that it improves things on the assumption that every label is correct, which is how these techniques are almost always evaluated and recommended.

This study doesn't propose a new, better correction method. Its contribution is an honest, statistically rigorous audit of the correction methods practitioners already use every day, run under conditions closer to how real, imperfect data actually behaves — with every result traceable back to a specific, reproducible number, and every method's implementation checked against its original source rather than assumed correct.

9. Glossary of Terms

Class imbalance

One category in your data is much rarer than the other(s).

Minority class

The rarer category (e.g. "fraud," "spam," "malignant").

Majority class

The more common category.

Label noise

Some of the labels in the dataset are simply wrong.

Directed label noise

Label errors that systematically flow in one particular direction rather than being evenly random.

In-context learning

A model that makes predictions by reading the training examples as part of its input "prompt," rather than being separately trained on them first.

Stratified split

Dividing data into train/test sets while preserving the same class ratio in both pieces.

Seed

A starting number for a random process, so the exact same "random" outcome can be reproduced later.

Balanced accuracy

The average of how accurate the model is on each class separately — prevents a model from looking good just by favoring the common class.

Break-even point (c^*)

The noise level at which a correction method stops being net-helpful and starts being net-harmful.

Oracle

A hypothetical version of a method that is (unrealistically) given the true, uncorrupted information it would not actually have access to in real deployment — used only to measure how much the corruption itself is costing.

Statistical significance

A measure of how likely it is that an observed difference is a real effect rather than random chance. A very small p-value means "very unlikely to be chance."